



Zambia University College of Technology

Advancing Knowledge and Innovation

NAMES:

MARTHA CHOOKA 2410008

PRINCE ZULU 2410237

BENNY SIMPAMBA 2410092

COURSE: BSE 2210

LECTURER: MR J. CHIKWANDA

PATTERNS

Software design patterns are reusable solutions to common problems that arise during software design. They capture proven development practices and principles that experienced developers have discovered and refined over time, providing standardized approaches to solving recurring design challenges. In this report we will take a look at some the patterns that can be used to create the application as per the requirements in the assignment documentation.

1. Observer Pattern

The Observer pattern, which acts as the *Notifier*, lets one object (called the *subject* or *publisher*) automatically notify many other objects (called *observers* or *subscribers*) whenever something happens — without having to know exactly who they are.

Upon having multiple users registering and notifications:

- The system can treat the user registration module as the subject.
- Notification services, admin dashboards, and analytics trackers are the observers.
- When a new user registers, the registration module “fires an event” → the observers automatically get notified and act (send push notification, update analytics, display in admin dashboard, etc.)

Example flow:

1. A new user registers.
2. The registration module triggers a “userRegistered” event.
3. All observers listening for that event (NotificationService, AnalyticsService, etc.) are automatically informed.
4. NotificationService instantly sends a push notification or email.

Advantages

- Notifications are instant and automatic.
- There is no need to tightly connect your registration code to notification logic (reduces coupling).

- Makes it easier to add more observers later (e.g., send SMS, log user trends) without changing the registration code.

2. Factory Pattern

The Factory pattern provides a central place to create objects, especially when you have many different types but want to hide the creation details from the rest of the program.

Upon having multiple users registering and notifications:

- There will be a need to send different types of notifications: email, SMS, mobile push, or in-app alerts.
- Instead of the main program having to know *how* to create each type of notification object, the use of a [NotificationFactory](#) that knows how to create the right one based on the situation, can be used.

Example flow:

1. A new user registers.
2. The system asks the [NotificationFactory](#) for the correct type of notification sender (e.g., [EmailNotifier](#), [PushNotifier](#), [SMSNotifier](#)).
3. The factory looks at user preference (e.g., push or email) and returns the right object.
4. The notification is sent immediately using the returned object.

Advantages

- Makes it easy to add new notification methods later (e.g., WhatsApp, in-app message) without changing existing code.
- Keeps code clean, flexible, and easy to maintain — modification of the factory only happens when there is introduction of a new notification type.
- Great for large-scale systems where many object types need to be created dynamically.

3. Strategy Pattern

The Strategy pattern lets you choose the best algorithm or method at runtime rather than hardcoding one way of doing something.

Upon having multiple users registering and notifications, different strategies maybe used for:

- Handling user registrations (e.g. standard form vs. OAuth login).
- Sending notifications (e.g. direct socket message for active users, push notification for offline ones).
- Managing load balancing (e.g., round-robin vs. priority-based distribution for many simultaneous users).

Example flow:

1. The system wants to send a notification to each user.
2. It checks the user's device status:
 - If online → use [WebSocketStrategy](#) for instant notification.
 - If offline → use [PushNotificationStrategy](#) for delayed delivery.
3. The strategy can change dynamically without changing the rest of the notification system.

Advantages

- Behavior can easily be adapted depending on conditions (like user online status or network load).
- Keeps the system extensible — new strategies can be added without rewriting the core logic.
- Makes it easy to test and optimize different behaviors independently.

In conclusion, the Observer Pattern is the best pattern to be used for the application mainly due to the following reasons:

- It is fundamentally designed for real-time, one-to-many communication. When a key event happens (like a user registering), the Observer pattern ensures that all interested services (push notification, email, SMS) are triggered immediately and in parallel. This is further emphasizes the 'without delay' notification nature.
- It can handle multiple users seamlessly and effortlessly. It doesn't matter if one user registers or a thousand. For each event, a single announcement is made, and all subscribed observers kick into action

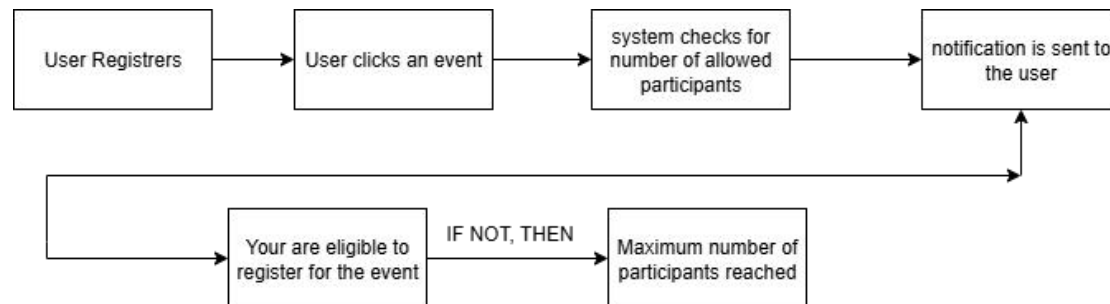
simultaneously. The source of the event doesn't get delayed by waiting for each notification task to finish.

- It creates a resilient architecture by decoupling the core action (user registration) from its side effects (sending notifications), preventing bottlenecks. If the email service is temporarily slow, it won't hold up the push notification service. The push notification can still be sent instantly.

DIAGRAMS

Behavioral Diagram

This shows what happens in a session for event registration for a single user and how and which notification is delivered to that user



Observer Design Pattern

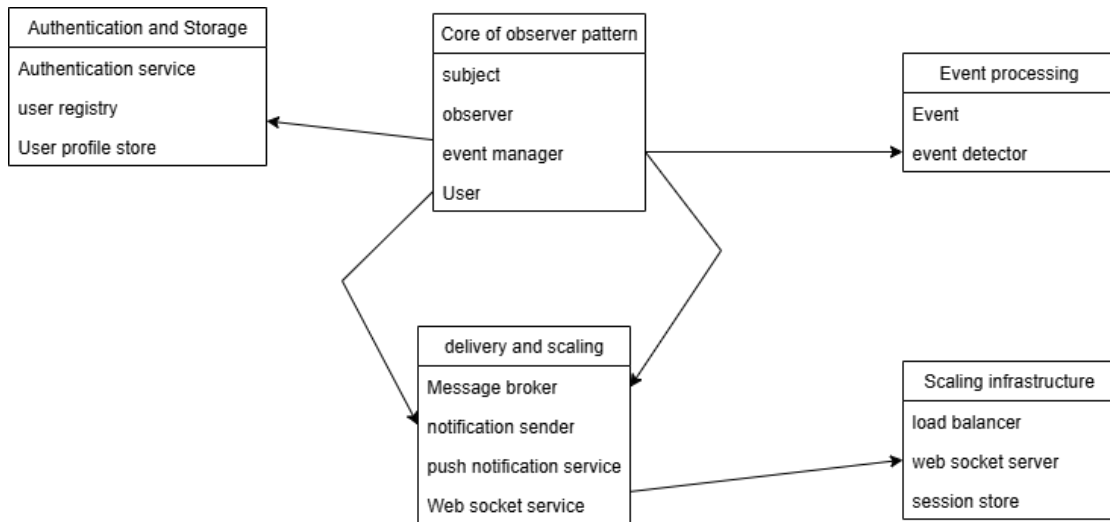
Authentication and Storage: These verifies the username and grants access to the application. It also stores all the registered users usernames.

Core: contains the subject to which the observer observes changes and adjusts accordingly

Event Processing: Connected to the event manager, Manages all events and changes that occur to them. It also detects when a new event is uploaded.

Delivery and Scaling: Contains the message broker which is the intermediary which allows communication between the publisher(observer) and the subscribers. Then notifications are sent to the subscribers concurrently.

Scaling Infrastructure: Contains the Load balancer which handles situations of multiple registrations. And also contains the web socket server to which the web socket service for online notifications is connected. A session store is also included which contains the session data, methods to observers to register and unregister themselves, and provides methods to notify all registered observers when the session data changes



Concurrency

Concurrency in software is the ability of a program to execute multiple tasks at the same time (or appear to do so). It doesn't always mean they literally run simultaneously on the CPU — instead, it means the tasks can **overlap in time**, improving efficiency and responsiveness.

How does one handle concurrency?

1. At the Database Level

- **Transactions** – Ensure operations like “register for event” are atomic (either fully succeed or fail).
- **Row-level locks** – Prevent two people from registering for the last available slot at the same time.
- **Optimistic Concurrency Control** – Let updates proceed, but reject if data changed since it was last read (forces user to refresh).

Example: If two people try to book the last seat in a workshop, only one transaction commits successfully.

2. At the Backend/API Level

- Use **message queues** (e.g., RabbitMQ, Kafka, or Redis Pub/Sub) to handle lots of concurrent updates without overwhelming the server.
- Apply **rate limiting & throttling** to prevent a single user from flooding the system.
- Use **async/await** or **non-blocking I/O** (Node.js, Spring Web Flux, etc.) so the server can handle thousands of concurrent requests.

3. Real-Time Updates for Users

- **web Sockets** – Keep a persistent connection open so updates (like “New event added” or “Workshop is full”) instantly push to users.
- **Server-Sent Events (SSE)** – Simple way to stream live updates one-way from server to browser.
- **Push Notifications (mobile)** – For instant alerts when a user is offline.

4. Concurrency Safety in Code

- Use **locks/mutexes** for shared resources (if you have in-memory counters).
- Prefer **stateless services** → scale easily across multiple servers.
- Use **background workers** to process heavy tasks (e.g., sending 100s of notifications) without blocking the main app.

5. Example Flow: User Registering for a Workshop

1. User clicks “Join Workshop.”
2. Backend starts a **transaction**:
 - Check if spots are available.
 - Deduct one spot.
 - Save the booking.
3. Commit transaction → only one user gets the last spot.
4. Trigger a **real-time update** over WebSocket:
 - Everyone's app updates instantly: “Workshop now full.”